

Title: DATA VORTEX '26 — PHASE 2 ANALYTICAL SQL LOGIC EXPLANATION

Event & Stage: Aaruush '26 | Round 1 (Phase 2)

Candidate Name: Gourab Saha

QUESTION 1

E3—Average Engagement by Platform

Challenge: Calculate the average likes, shares, and comments for each platform. Which platform generates the highest average total engagement?

LOGIC & APPROACH DOCUMENTATION

Challenge Definition

The goal is to calculate the average volume of interactions across individual engagement vectors (`likes`, `shares`, `comments`) for each social platform and identify which channel commands the highest overall engagement intensity.

Technical Architecture & Design Decisions

- **Defensive Arithmetic via `COALESCE` Handling:**
Standard aggregate functions like `AVG()` in SQL automatically ignore `NULL` values. However, direct scalar addition across raw metric columns (e.g., `likes + shares + comments`) evaluates to `NULL` if even a single metric is missing. To prevent silent data dropped rows and skewed averages, each column is wrapped in `COALESCE(column, 0)`. This guarantees mathematical stability across incomplete records.
- **CTEs for Execution Clarity:**
The logic uses a Common Table Expression (`PlatformMetrics`) to encapsulate raw aggregation before applying final formatting operations like `ROUND(..., 2)`. This decouples heavy aggregation from representation layer formatting, keeping execution plans clean and readable.
- **String Trimming & Categorical Filtering:**
A basic `IS NOT NULL` check often fails when dealing with dirty datasets containing whitespace-only strings. The filter `WHERE platform IS NOT NULL AND TRIM(platform) != ''` ensures string hygiene before grouping operations begin.

- **Precision Alignment:**

Calculated averages are explicitly cast using `ROUND(..., 2)` in the final selection projection to prevent floating-point representation artifacts in reported statistics.

QUESTION 2

M1 — Which Locations Generate the Most Engagement? Challenge:

Using both datasets, calculate the total engagement generated by users from each location. Return the location, number of posts, and total engagement. Rank locations from highest to lowest engagement

LOGIC & APPROACH DOCUMENTATION

Challenge Definition

The objective is to synthesize user location metadata with activity streams to evaluate aggregate geographic engagement, calculate post volumes per market, and establish a ranked distribution of regional performance.

Technical Architecture & Design Decisions

- **Relational Key Join Integrity (INNER JOIN):**

The logic joins `users` and `posts` on the primary-foreign key relationship `u.user_id = p.user_id`. An `INNER JOIN` ensures that calculations only consider activity tied to valid user profiles, preventing orphan post records from polluting geographic totals.

- **Null-Safe Attribute Aggregation:**

Missing location attributes are normalized using `COALESCE(u.location, 'Unknown')` within both the projection and the `GROUP BY` clause. Engagement components (`likes`, `shares`, `comments`) are individually wrapped in `COALESCE` prior to summation to prevent missing post metrics from nullifying valid interaction counts.

- **Dense Window Ranking (DENSE_RANK):**

Unlike standard `RANK()`—which leaves gaps in ordinal sequences during tied values—or simple `ORDER BY` sorting, `DENSE_RANK() OVER (ORDER BY total_engagement DESC)` provides deterministic ranking across geographic markets without skipping rank index numbers if score ties occur.

- **Two-Pass CTE Architecture:**

Aggregation logic is separated into a Common Table Expression (`LocationEngagement`), isolating heavy relational joins and group sums from the final

projection and windowing functions. This design allows query engines to optimize aggregation memory allocation prior to applying window rank calculations.

QUESTION 3

H4 — Follower to Engagement Anomaly Challenge: Find users who have fewer than 5,000 followers but whose total post engagement places them among the top 10% of all users. These users may represent unusually high performing or suspicious accounts. This requires multiple levels of analysis.

LOGIC & APPROACH DOCUMENTATION

Challenge Definition

The objective is to execute a multi-tiered anomaly detection analysis that isolates low-audience accounts (<5,000 followers) producing disproportionately massive platform reach—specifically placing within the top 10% (1st decile) of overall user engagement. These outliers represent either hyper-viral micro-creators or synthetic engagement farming vectors.

Technical Architecture & Design Decisions

- **User-Level Multi-Vector Aggregation (UserAggregates CTE):**
The primary stage connects `users` and `posts` using an `INNER JOIN` on `user_id`. It sums interaction vectors (`likes`, `shares`, `comments`) guarded by `COALESCE(metric, 0)` to construct a clean aggregate score (`total_user_engagement`) alongside posting frequency (`total_posts`) per user.
- **Statistical Decile Segmentation via NTILE(10) (RankedUsers CTE):**
To dynamically calculate the top 10% cutoff without hardcoding arbitrary static thresholds or using non-portable vendor functions, the query applies the `NTILE(10)` window function ordered by `total_user_engagement DESC`. This partitions the entire populated user base into 10 equal buckets, where `engagement_decile = 1` mathematically isolates the top 10th percentile.
- **Multi-Filter Anomaly Thresholding:**
The final outer query applies a dual-predicate filter:
 1. `follower_count < 5000`: Constrains the audience dimension to nano/micro accounts.
 2. `engagement_decile = 1`: Restricts results to top-decile engagement performers.
- **Two-Stage CTE Modular Design:**
Structuring the query into two distinct abstraction layers (`UserAggregates` $\rightarrow$ `RankedUsers`) prevents windowing evaluation errors. Because window functions like

NTILE cannot be directly evaluated inside a standard SQL **WHERE** clause without a subquery or CTE, this architecture ensures full ANSI-SQL compliance and optimization.